

Huffman 부호 (Huffman Code) 원리

1. 개요

Huffman 부호는 1952년 David A. Huffman이 제안한 **무손실 압축** 알고리즘으로, 정보이론의 엔트로피 개념을 활용하여 **자주 나타나는 심볼에는 짧은 코드, 드물게 나타나는 심볼에는 긴 코드**를 할당함으로써 평균 코드 길이를 최소화합니다.

- **특징**: 접두 부호(prefix-free code), 즉 어떤 코드워드도 다른 코드워드의 접두어가 되지 않아 **유일 복호화**가 가능합니다.
- **응용**: ZIP, JPEG, MP3 등 많은 압축 표준의 기반이 됩니다.

2. 정보이론적 배경

2.1 엔트로피와 평균 코드 길이

이산 메모리 없는 정보원 X 의 **엔트로피**는 다음과 같습니다.

$$H(X) = - \sum_i p(x_i) \log_2 p(x_i) \quad [\text{bits}]$$

Shannon의 제1 부호화 정리에 따르면, 어떤 유일 복호화 가능한 부호도 그 **평균 코드 길이 $\bar{L}$ 가 엔트로피보다 작을 수 없습니다.**

$$\bar{L} \geq H(X)$$

Huffman 부호는 $\bar{L}$ 을 $H(X)$ 에 **매우 가깝게** 만드는 실현 가능한 부호입니다. (각 심볼의 코드 길이가 $-\log_2 p(x_i)$ 에 가깝게 됨)

2.2 접두 부호 (Prefix-Free Code)

- **접두 부호**: 어떤 코드워드도 다른 코드워드의 **앞부분(접두어)**가 되지 않는 부호.
- 예: 0, 10, 110, 111 → 접두 부호 ✓
0, 01, 1 → 0 이 01의 접두어이므로 접두 부호 X
- 접두 부호는 **비트 스트림을 왼쪽에서 오른쪽으로** 스캔하면서 유일하게 복호화할 수 있습니다.

3. Huffman 부호화 알고리즘

3.1 핵심 아이디어

빈도가 낮은 심볼일수록 긴 코드, 빈도가 높은 심볼일수록 짧은 코드를 주어
평균 코드 길이 $\bar{L} = \sum_i p(x_i) \cdot \ell_i$ 를 최소화합니다.

3.2 Huffman 트리 구성 (알고리즘)

1. **입력**: 각 심볼 x_i 와 그 빈도(또는 확률) f_i .
2. **초기화**: 각 심볼을 **리프 노드**로 하는 1개 노드짜리 트리들의 집합.
3. **반복** (노드가 1개 남을 때까지):
 - 빈도가 가장 작은 두 트리 T_1, T_2 를 선택.
 - 두 트리를 **자식**으로 하는 새로운 루트 노드를 만들고, 그 빈도는 $f_1 + f_2$.
 - 새 트리를 집합에 넣고, T_1, T_2 는 집합에서 제거.
4. **결과**: 남은 하나의 트리가 **Huffman 트리**입니다.
5. **코드 할당**: 루트에서 각 리프까지 가는 경로에서 왼쪽 자식은 **0**, 오른쪽 자식은 **1**로 두고, 경로의 0/1 열이 해당 심볼의 **코드워드**가 됩니다.

이렇게 하면 자동으로 **접두 부호**가 되고, **평균 코드 길이가 최소인** 부호를 얻습니다.

3.3 간단한 예시

심볼	빈도
A	45
B	13
C	12
D	16
E	9
F	5

- 가장 작은 두 개: F(5), E(9) → 합 14인 노드 생성.
- 그다음 작은 두 개: C(12), (F+E)(14) 또는 B(13) 등 순서대로 합치면 됩니다.

최종적으로 예를 들어 다음과 같은 코드가 얻어질 수 있습니다.

- A: 0
- B: 101
- C: 100
- D: 111
- E: 1101
- F: 1100

(같은 빈도에서 왼쪽/오른쪽 선택에 따라 코드가 달라질 수 있으나, 평균 길이는 동일합니다.)

4. 평균 코드 길이와 압축률

- 고정 길이 부호 (예: 6심볼이면 3비트): $\bar{L}_{\text{fixed}} = \lceil \log_2 6 \rceil = 3 \text{ bits/symbol}$.
- Huffman 부호: $\bar{L}_{\text{Huffman}} = \sum_i p_i \ell_i$.
- 압축률: 원본 비트 수 대비 부호화 후 비트 수의 비율로 정의할 수 있으며, $\bar{L}$ 이 작을수록 압축이 잘 됩니다.

$$\bar{L} \approx H(X) \quad \Rightarrow \quad \text{엔트로피에 근접한 효율}$$

5. 실습에서 확인할 내용

1. 빈도 분포를 입력하여 Huffman 트리를 구성하고, 각 심볼의 **코드워드**와 **코드 길이**를 출력.
2. 평균 코드 길이 $\bar{L}$ 과 엔트로피 $H(X)$ 를 계산하여 $\bar{L} \geq H(X)$ 및 $\bar{L} \approx H(X)$ 확인.
3. 문자열 인코딩/디코딩: 주어진 텍스트를 Huffman 부호로 압축하고, 비트열만으로 다시 복원하는 과정 구현.

6. 참고

- Huffman 부호는 **2진 트리** 기반이며, 3진 이상의 부호로 확장할 수 있습니다.
- **적응형(Adaptive) Huffman**은 빈도를 실시간으로 갱신하며 부호를 바꿀 수 있습니다.
- **실제 압축**에서는 Huffman 테이블(심볼-코드워드 쌍)을 파일에 함께 저장하거나, 미리 정의된 테이블을 사용합니다.

이 자료와 함께 제공되는 `huffman.py` 실습 프로그램으로 위 내용을 직접 구현하고 실험해 볼 수 있습니다.